

Project: Automated Inventory & PO System (Object-Oriented Approach)

1. System Description

1.1 Purpose of the System

The Automated Inventory & PO System is designed strictly using Object-Oriented Analysis and Design (OOAD) principles to manage warehouse inventory and monitor sales in real-time. By representing system entities as interacting objects (e.g., User, Product, PurchaseOrder), the system aims to encapsulate business logic, eliminate human errors in manual stocktaking, and prevent sudden product shortages. When a Product object's stock state reaches its predefined "Reorder Point", the system instantiates a new PurchaseOrder object, routes it to a FinanceManager object for approval, and ultimately transmits it to an external SupplierAPI interface.

1.2 Object-Oriented Architectural Principles Applied

Encapsulation: All sensitive data, such as product quantities and financial approval limits, are hidden within their respective classes (private attributes) and can only be accessed or modified through secure public methods (getters/setters).

Inheritance: A base abstract class User is created containing common attributes (ID, Name, Email, Login method). Specific actors like InventoryManager and FinanceManager inherit from this base class, adding their specific roles and permissions.

Polymorphism: The system utilizes method overriding and interfaces, specifically for the SupplierAPI, allowing the system to communicate with multiple different external suppliers using a unified method signature like transmitPO().

1.3 Functional Requirements (FR)

User & Authentication Management

FR1: The system shall instantiate specific subclass objects (InventoryManager, FinanceManager, SystemAdmin) upon successful secure authentication, inheriting from the base User abstract class.

FR2: The system shall allow the SystemAdmin object to instantiate new user profiles and assign specific roles.

FR3: The system shall securely encapsulate user passwords and verify them using the login() method within the User class.

FR4: The system shall provide a logout() method to securely terminate the user session and clear objects from active memory.

Product & Inventory Control

FR5: The system shall allow the InventoryManager to instantiate new Product objects and persist them in the database.

FR6: The system shall allow the InventoryManager to invoke setter methods to mutate the reorderPoint and maxStock attributes for each Product object.

FR7: The system shall strictly encapsulate the currentStock attribute, only allowing modifications through the updateStock() public method.

FR8: The Product object shall dynamically compute the required order quantity by invoking the calculateRequiredQuantity() method (Max Stock - Current Stock).

FR9: The system shall trigger the isLowStock() method automatically upon every sales transaction to evaluate the product's state.

Purchase Order (PO) Automation

FR10: The system shall automatically instantiate a new PurchaseOrder object when the isLowStock() method evaluates to True.

FR11: The system shall assign an initial state of "Draft" to any newly instantiated PurchaseOrder object.

FR12: The system shall strictly prevent the instantiation of a new PurchaseOrder for a specific Product if an existing order object is already in a "Draft" or "Pending Approval" state (Duplicate Prevention).

FR13: The system shall allow the FinanceManager to retrieve an array/list of all PurchaseOrder objects filtered by their current state.

Financial Governance & Workflow

FR14: The system shall restrict the invocation of the approvePO() and rejectPO() methods exclusively to instances of the FinanceManager class.

FR15: The system shall mandate the input of a "reason" string argument when the FinanceManager invokes the rejectPO() method.

FR16: The system shall mutate the PurchaseOrder object's state to "Approved" or "Rejected" based on the FinanceManager's method invocation.

FR17: The system shall notify the FinanceManager object instantly when a new PurchaseOrder object is instantiated and requires approval.

API Integration

FR18: The system shall invoke the transmitPO() method of the SupplierAPI interface immediately after a PurchaseOrder object's state mutates to "Approved".

FR19: The SupplierAPI interface shall parse the external API response to update the PurchaseOrder state to "Transmitted/Sent" or "Failed".

FR20: The system shall trigger a retry mechanism if the transmitPO() method throws a network timeout exception.

1.4 Non-Functional Requirements (NFR)

Performance & Efficiency

NFR1: Object state updates and method executions for stock adjustments must complete in less than 2 seconds.

NFR2: The system shall support concurrent method invocations by at least 150 User objects simultaneously without noticeable performance degradation.

NFR3: The retrieval of PurchaseOrder object lists for the FinanceManager dashboard must load within 3 seconds.

Security & Privacy

NFR4: Passwords encapsulated within the User object must be hashed using industry-standard algorithms (e.g., SHA-256) before persistence in the database.

NFR5: The system must enforce Role-Based Access Control (RBAC) at the method invocation level, completely denying unauthorized calls to sensitive financial methods.

NFR6: All API transmissions payload via the SupplierAPI interface must be encrypted using TLS 1.2 or higher protocols.

NFR7: The system shall maintain an immutable Audit Log object to record the User ID and timestamp for every state mutation in a PurchaseOrder.

Reliability & Availability

NFR8: The automated event listener that triggers PO instantiation must have a guaranteed availability of 99.9% to prevent critical stockouts.

NFR9: The system shall implement robust exception handling (Try-Catch blocks) to prevent application crashes when the SupplierAPI experiences external downtime.

NFR10: Automated database backups of all serialized objects must occur every 24 hours to ensure disaster recovery.

Scalability & Maintainability

NFR11: The SupplierAPI must be strictly designed as an Interface (or Abstract Class) to support Polymorphism, allowing easy addition of new supplier subclasses in the future without altering core logic.

NFR12: The database schema and object relational mapping (ORM) must be scalable to persist up to 100,000 Product objects and 500,000 PurchaseOrder records without architectural changes.

NFR13: The system architecture must strictly adhere to SOLID object-oriented principles to ensure high maintainability by future development teams.

Usability

NFR14: The user interface interacting with the backend objects must feature a responsive design, fully compatible with both desktop screens and mobile devices.

NFR15: Exceptions thrown by backend objects (e.g., invalid input for Reorder Point) must be caught and displayed to the UI as clear, user-friendly error messages, hiding technical stack traces.

2. Actors Identification (Classes Mapping)

Inventory Manager (Primary Actor / Subclass): Inherits from User. Responsible for invoking methods to set Reorder Points and update Product states.

Finance Manager (Primary Actor / Subclass): Inherits from User. Responsible for invoking approval/rejection methods on PurchaseOrder objects.

System Administrator (Secondary Actor / Subclass): Inherits from User. Manages system configurations and User object instantiations.

System Event Listener (Automated Actor): An internal background process that continuously evaluates Product states and triggers events.

Supplier API (External Actor / Interface): An external interface that receives serialized PurchaseOrder objects and returns acknowledgment states.

3. Detailed Use Case Descriptions

Use Case 1: Set Reorder Point & Stock Limits (UC-01)

Element	Description
Use Case ID	UC-01
Use Case Name	Set Reorder Point & Stock Limits
Primary Actor	Inventory Manager (Object Instance)
Description	The Inventory Manager invokes setter methods to update the minimum and

	maximum threshold attributes of a Product object.
Pre-conditions	1. InventoryManager object is authenticated. 2. Target Product object exists in memory/database.
Post-conditions	The Product object's attributes (reorderPoint, maxStock) are mutated and persisted.
Main Success Scenario	1. Inventory Manager requests Product object details. 2. System returns the current state of the Product. 3. Manager inputs new threshold values. 4. System invokes product.setReorderPoint() and validates input. 5. State is saved and success message is returned.

Use Case 2: Auto-Instantiate Draft Purchase Order (UC-02)

Element	Description
Use Case ID	UC-02
Use Case Name	Auto-Instantiate Draft PO
Primary Actor	System (Event Listener Object)
Secondary Actor	Finance Manager
Description	The system detects a state change in a Product object where current stock $\leq$ reorder point, triggering the

	instantiation of a PurchaseOrder object.
Pre-conditions	Product object's isLowStock() method returns True.
Post-conditions	A new PurchaseOrder object is created with state "Draft".
Main Success Scenario	1. Sales transaction invokes product.updateStock(). 2. System evaluates product.checkStockLevel(). 3. If threshold met, system calculates required quantity. 4. System executes new PurchaseOrder(), passing Product details. 5. System notifies the FinanceManager object of the new pending instance.

Use Case 3: Review and Transmit Purchase Order (UC-03)

Element	Description
Use Case ID	UC-03
Use Case Name	Review and Transmit PO
Primary Actor	Finance Manager
Secondary Actor	Supplier API
Description	The Finance Manager invokes the approve method on a PO object, which triggers API transmission.
Pre-conditions	PurchaseOrder object exists with state "Draft".

Post-conditions	PurchaseOrder state mutates to "Approved" and then "Sent".
Main Success Scenario	1. Finance Manager retrieves list of Draft PurchaseOrder objects. 2. Manager selects an object and reviews its attributes. 3. Manager invokes po.approve(). 4. System changes PO state to "Approved". 5. System invokes supplierAPI.transmitPO(this). 6. Supplier API returns success acknowledgment. PO state updates to "Sent".

Use Case 4: Receive PO Shipment & Mutate Stock (UC-04)

Element	Description
Use Case ID	UC-04
Use Case Name	Receive PO Shipment & Mutate Stock
Primary Actor	Inventory Manager
Description	The Inventory Manager confirms the physical arrival of goods, invoking methods to update inventory levels and close the order.
Pre-conditions	PurchaseOrder state must be "Sent" or "Acknowledged".
Post-conditions	Product object's currentStock is increased, and PurchaseOrder state mutates to "Closed".

Main Success Scenario	1. Inventory Manager retrieves the specific PurchaseOrder object. 2. Manager verifies received quantities. 3. Manager invokes po.receiveShipment(). 4. System invokes product.updateStock() to add the new items. 5. System updates PO state to "Closed".
------------------------------	---

Use Case 5: Instantiate New Product Object (UC-05)

Element	Description
Use Case ID	UC-05
Use Case Name	Instantiate New Product Object
Primary Actor	Inventory Manager
Description	The Inventory Manager creates a new item record in the system's database.
Pre-conditions	Inventory Manager has valid write permissions.
Post-conditions	A new Product object is persisted in the database.
Main Success Scenario	1. Manager inputs product details (SKU, Name, Price). 2. System invokes constructor new Product(). 3. System saves the object and returns a success confirmation.

Use Case 6: Reject Purchase Order (UC-06)

Element	Description
Use Case ID	UC-06
Use Case Name	Reject Purchase Order
Primary Actor	Finance Manager
Description	The Finance Manager evaluates a draft PO and decides to decline it, preventing transmission to the supplier.
Pre-conditions	PurchaseOrder exists with state "Draft".
Post-conditions	PurchaseOrder state mutates to "Rejected".
Main Success Scenario	1. Manager retrieves Draft PurchaseOrder. 2. Manager selects the reject option and provides a String reason argument. 3. System invokes po.rejectPO(reason). 4. System updates state to "Rejected" and notifies Inventory Manager.

Use Case 7: Instantiate User Profiles (UC-07)

Element	Description
Use Case ID	UC-07
Use Case Name	Instantiate User Profiles
Primary Actor	System Administrator

Description	The Admin creates new user accounts, assigning them specific subclass roles (e.g., Finance, Inventory).
Pre-conditions	Admin object is securely authenticated.
Post-conditions	A new Subclass User object (e.g., FinanceManager) is created and stored.
Main Success Scenario	1. Admin inputs user details and selects role type. 2. System invokes admin.assignRole(). 3. System instantiates the chosen User subclass object. 4. Object is saved to the database.

Use Case 8: Audit System Logs (UC-08)

Element	Description
Use Case ID	UC-08
Use Case Name	Audit System Logs
Primary Actor	System Administrator
Description	Admin reviews immutable logs of object state mutations for security tracking.
Pre-conditions	System contains logged events.
Post-conditions	Log data is successfully retrieved and displayed.
Main Success Scenario	1. Admin requests to view system logs. 2. System retrieves Array of AuditLog objects. 3. Admin filters logs by date or User ID.

	4. System displays the formatted log history.
--	---

Use Case 9: Acknowledge PO Receipt (UC-09)

Element	Description
Use Case ID	UC-09
Use Case Name	Acknowledge PO Receipt
Primary Actor	Supplier API
Description	The external API returns a callback confirming the PO was successfully parsed by their system.
Pre-conditions	System has successfully executed transmitPO().
Post-conditions	PurchaseOrder state mutates to "Acknowledged".
Main Success Scenario	1. Supplier API receives payload. 2. Supplier API validates data and returns HTTP 200 OK. 3. System parses response. 4. System invokes po.changeState("Acknowledged").

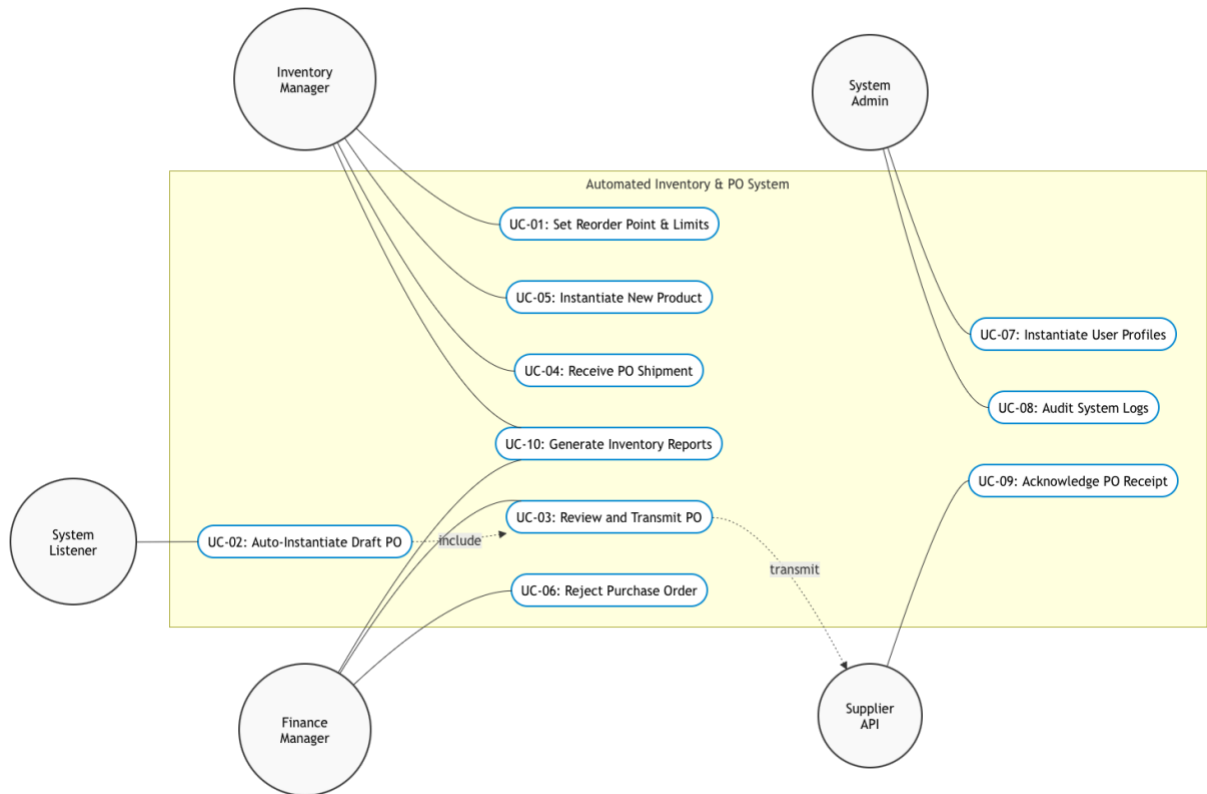
Use Case 10: Generate Inventory Reports (UC-10)

Element	Description
Use Case ID	UC-10
Use Case Name	Generate Inventory Reports
Primary Actor	Inventory Manager / Finance Manager

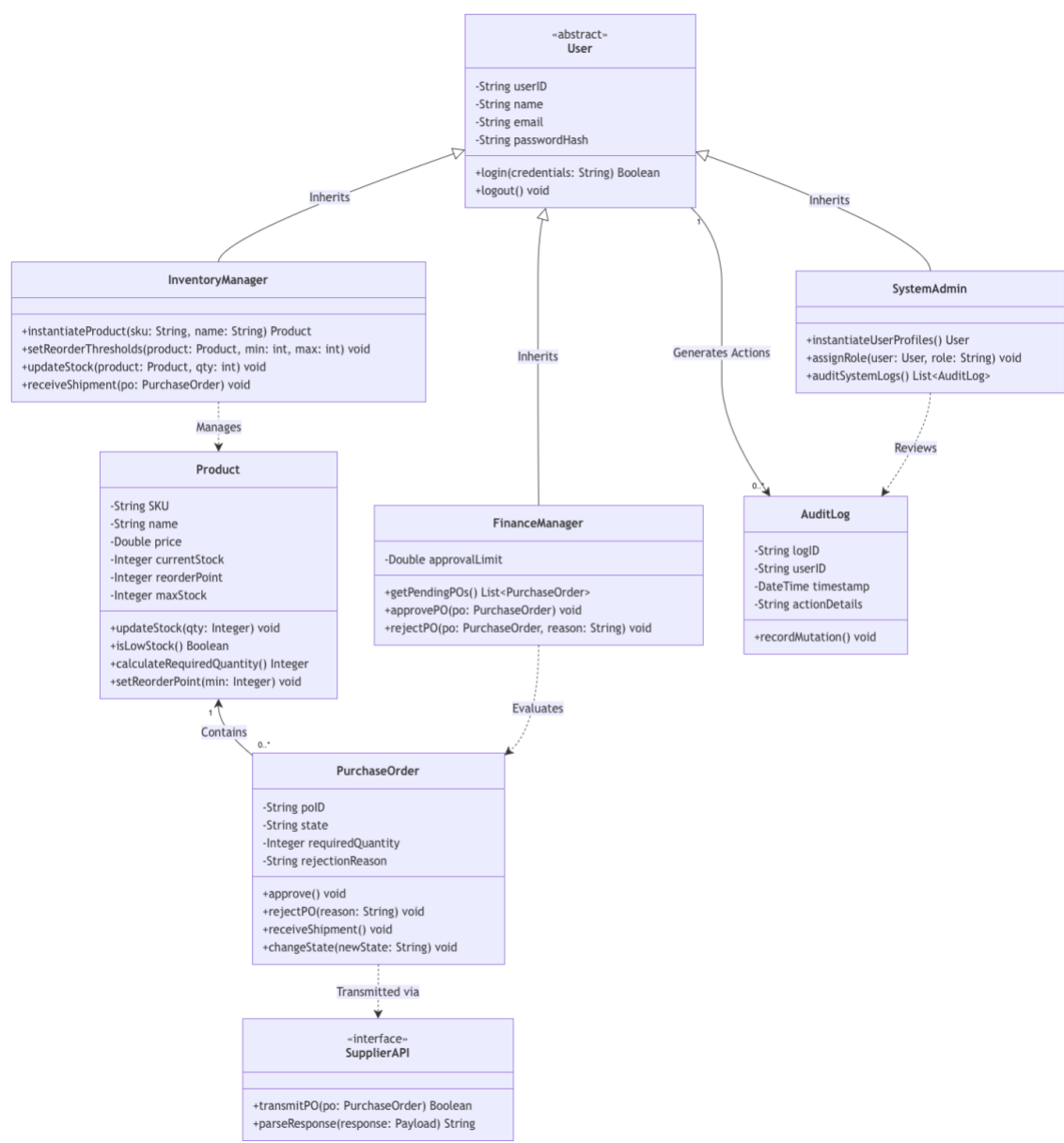
Description	The manager extracts data from multiple Product and PurchaseOrder objects to create a business summary.
Pre-conditions	Manager is authenticated.
Post-conditions	A formatted report object is presented to the UI.
Main Success Scenario	1. Manager requests a specific report type (e.g., PO History). 2. System iterates through relevant objects in the database. 3. System aggregates quantities and total costs. 4. System renders report data to the screen.

4. System Modeling Diagrams

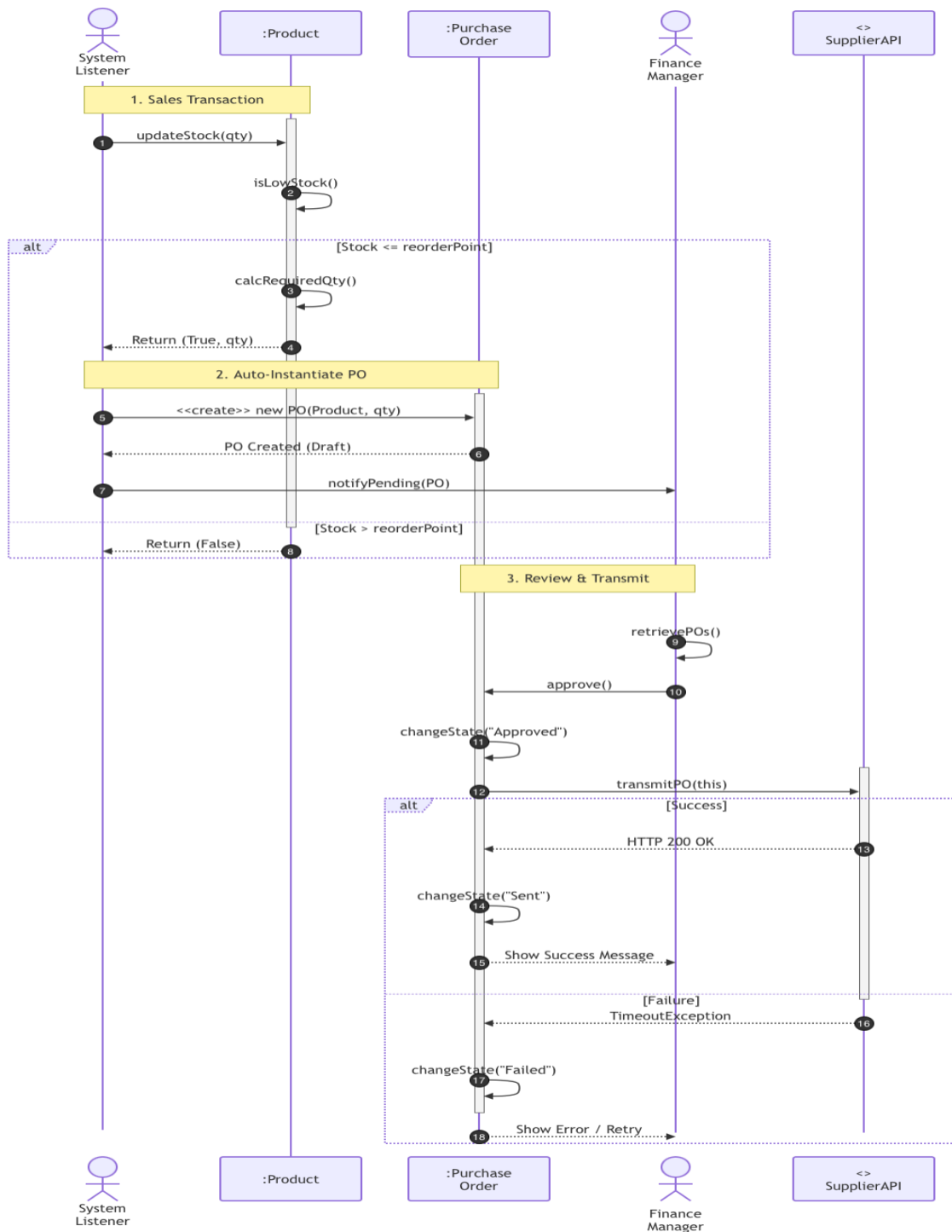
4.1 UML Use Case Diagram



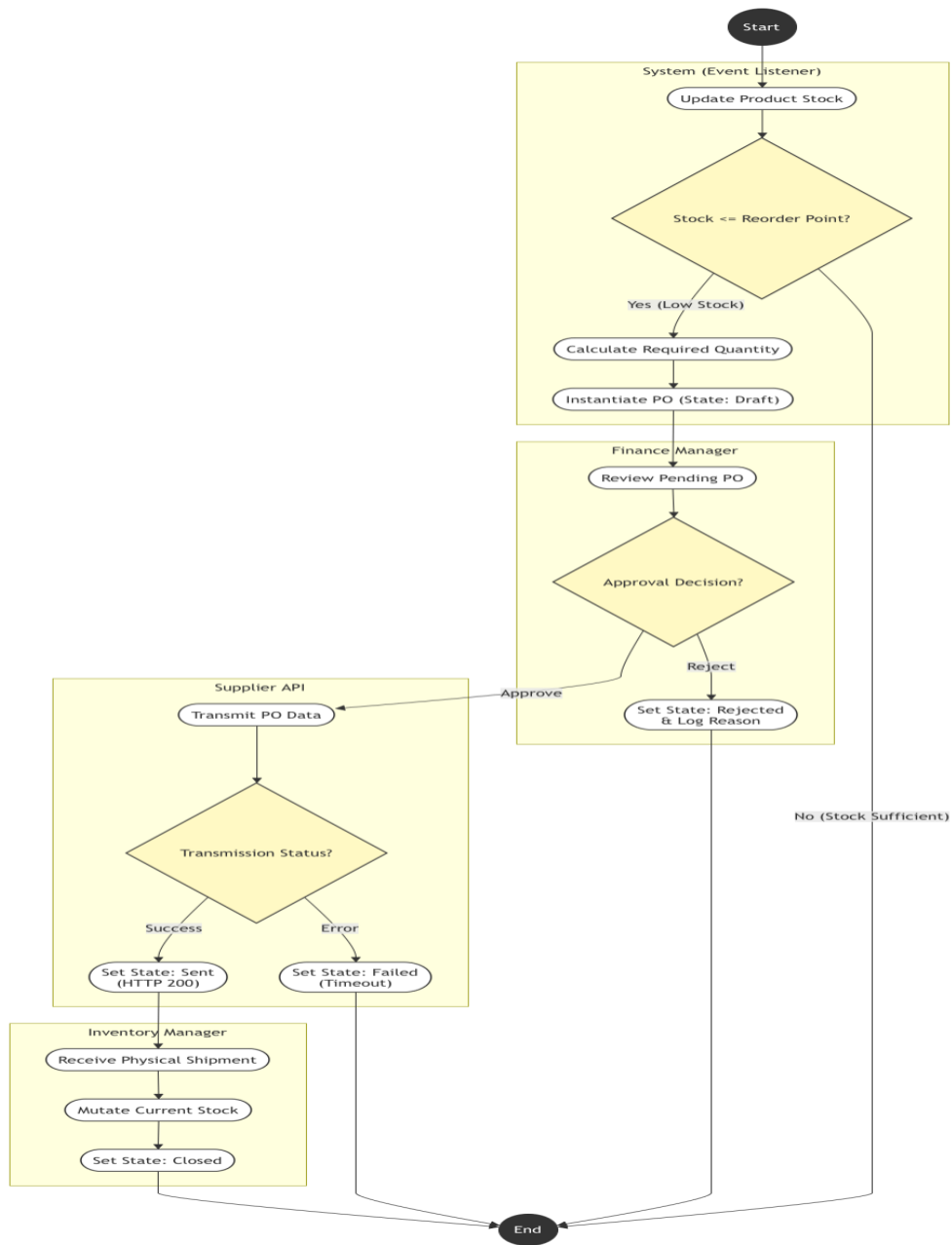
4.2 UML Class Diagram (Core OOP Structure)



4.3 UML Sequence Diagram



4.4 UML Activity Diagram



5. Conclusion

The transition to an Object-Oriented Analysis and Design (OOAD) approach for the Automated Inventory & PO System ensures a robust, modular, and scalable architecture. By clearly defining boundaries through encapsulation, establishing role hierarchies via inheritance, and standardizing external communications through polymorphic interfaces, the system accurately mirrors real-world business entities. This design not only streamlines the automated purchasing workflow but also provides a highly maintainable codebase that can effortlessly adapt to future technological and operational expansions.